Playwright Automation Framework Documentation

This document provides a detailed explanation of the Playwright Automation Framework developed for the Coursera website.

🚀 Approach

The framework is built using **TypeScript** and the **Page Object Model (POM)**. The primary goal is to create a clean, maintainable, and beginner-friendly automation suite that is easy to explain in an interview.

Key Design Principles:

- **Simplicity**: Avoided complex logic and over-engineering.
- **Readability**: Used meaningful variable names and detailed comments.
- **Modularity**: Separated locators, page actions, and test logic.
- **Robustness**: Included screenshot utilities and error handling.

📌 Scenarios

Scenario 1: Web Development Courses

- **Objective**: Automate the search and filtering of Web Development courses.
- **Key Actions**:
  - Navigate to `coursera.org`.
  - Search for “Web Development”.
  - Apply filters for “Beginner” level and “English” language.
  - Extract and print details (Name, Hours, Rating) of the first two courses.

Scenario 2: Language Learning Section

- **Objective**: Extract structured data from the Language Learning section.
- **Key Actions**:
  - Navigate to the Language Learning section.
  - Extract all available languages and levels.
  - Print the count and names of each in a structured format.

Scenario 3: Form Validation

- **Objective**: Validate form error handling for invalid input.
- **Key Actions**:
  - Navigate to the “For Enterprise” and “Courses for Campus” sections.
  - Fill out the “Ready to transform” form with an invalid email.
  - Capture and print the resulting error message.

💻 Code Explanation

1. Page Object Model (POM)

We use three main Page classes to represent different parts of the website:

- **HomePage.ts**: Handles navigation, search, and links to other sections.
- **CoursesPage.ts**: Handles filtering and data extraction from course listings.
- **EnterprisePage.ts**: Handles form filling and error message capture.

2. External Locators

All CSS and XPath selectors are stored in `locators/locators.ts`. This ensures that if a selector changes on the website, we only need to update it in one file, rather than searching through multiple Page classes.

3. Screenshot Utility

The `utils/screenshot-util.ts` class provides a reusable method to take timestamped screenshots. This is crucial for debugging and providing visual evidence of test execution.

4. Playwright Configuration

The `playwright.config.ts` file defines the test environment, including the base URL, browser settings (Chromium), and reporting options.

🎯 Interview Tips

When explaining this framework in an interview, focus on:

- **Why POM?**: Explain how it improves maintainability and readability.
- **Why TypeScript?**: Mention type safety and better IDE support.
- **Why External Locators?**: Highlight the ease of maintenance.
- **Error Handling**: Show how the framework captures error messages and takes screenshots on failure.

This framework is designed to demonstrate proficiency in modern automation practices while remaining accessible and easy to teach.